Lecture 9

Sorting in Linear Time

Slides are based on the textbook and its notes

Overview

- How fast we sort?
  - We will prove a lower bound, then beat it by playing a different game.

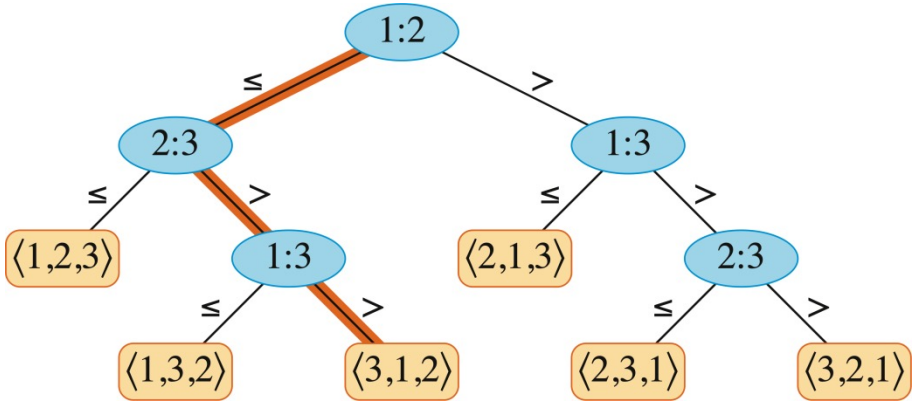
- Comparison sorting
  - The only operation that may be used to gain order information about a sequence is to compare pairs of elements.
  - Given two elements a_i and a_j, it performs one of the tests $a_i < a_j$, $a_i \leq a_j$, $a_i = a_j$, $a_i \geq a_j$, or $a_i > a_j$ to determine their relative order. We assume that all comparison have the form $a_i \leq a_j$ because they yield identical information about the relative order of a_i and a_j.
  - All sorts seen so far are comparison sorts: insertion sort, selection sort, merge sort, quicksort, and heapsort.

Lower bounds for sorting

- **Lower bounds**
  - All sorts seen so far are $\Omega(n \lg n)$.
  - We will show that $\Omega(n \lg n)$ is a lower bound for comparison sorts.
- **Decision tree**
  - Abstract of any comparison sort.
  - Represents comparisons made by
    - a specific sorting algorithm
    - on inputs of a given size.
  - Abstracts away everything else: control and data movement.
  - We are counting only comparisons.

Lower bounds for sorting: Decision tree

- ***Example***: The decision tree for insertion sort operating on three elements.



- An internal node (shown in blue) annotated by $i{:}j$ indicates a comparison between a_i and a_j. A leaf annotated by the permutation $\langle\pi(1), \pi(2), \ldots, \pi(n)\rangle$ indicates the ordering $a_{\pi(1)} \leq a_{\pi(1)} \leq \ldots \leq a_{\pi(n)}$. The highlighted path indicates the decisions made when sorting the input sequence $\langle a_1 = 6, a_2 = 8, a_3 = 5\rangle$. Going left from the root node, labeled 1:2, indicates that $a_1 \leq a_2$. Going right from the node labeled 2:3 indicates that $a_2 > a_3$. Going right from the node labeled 1:3 indicates that $a_1 > a_3$. Therefore, we have the ordering $a_3 \leq a_1 \leq a_2$, as indicated in the leaf labeled $\langle 3, 1, 2\rangle$. Because the three input elements have 3! = 6 possible permutations, the decision tree must have at least 6 leaves.

Lower bounds for sorting

- ***Theorem***
  - Any decision tree that sorts n elements has height $\Omega(n \lg n)$. Thus all comparison sort algorithm requires $\Omega(n \lg n)$ comparisons in the worst case.
- ***Proof***
  - Let the decision tree have height h and l reachable leaves.
  - $l \geq n!$
  - $l \leq 2^h$ (see Section B.5.3: in a complete k-ary tree, there are k^h leaves)
  - $n! \leq l \leq 2^h$ or $2^h \geq n!$
  - Take logarithms: $h \geq \lg(n!)$
  - Use Stirling's approximation: $n! > (n/e)^n$ (by equation (3.23))

    $h \geq \lg(n/e)^n = n \lg(n/e) = n \lg n - n \lg e = \Omega(n \lg n)$
- ***Corollary***
  - Heapsort and merge sort are asymptotically optimal comparison sorts.

Sorting in linear time: Counting sort

- We will discuss several non-comparison sorts.

- **Counting sort**
  - Depends on a key assumption: numbers to be sorted are integers in $\{0, 1, \ldots, k\}$.
  - **Input**: $A[1 : n]$, where $A[j] \in \{0, 1, \ldots, k\}$ for $j = 1, 2, \ldots, n$. Array A and values n and k are given as parameters.
  - **Output**: $B[1 : n]$, sorted.
  - **Auxiliary storage**: $C[0 : k]$.

Sorting in linear time: Counting sort

```
COUNTING-SORT(A, n, k)
 1  let B[1 : n] and C[0 : k] be new arrays
 2  for i = 0 to k
 3      C[i] = 0
 4  for j = 1 to n
 5      C[A[j]] = C[A[j]] + 1
 6  // C[i] now contains the number of elements equal to i.
 7  for i = 1 to k
 8      C[i] = C[i] + C[i − 1]
 9  // C[i] now contains the number of elements less than or equal to i.
10  // Copy A to B, starting from the end of A.
11  for j = n downto 1
12      B[C[A[j]]] = A[j]
13      C[A[j]] = C[A[j]] − 1    // to handle duplicate values
14  return B
```

Sorting in linear time: Counting sort

```
COUNTING-SORT(A, n, k)
 1  let B[1 : n] and C[0 : k] be new arrays
 2  for i = 0 to k
 3      C[i] = 0
 4  for j = 1 to n
 5      C[A[j]] = C[A[j]] + 1
 6  // C[i] now contains the number of elements equal to i.
 7  for i = 1 to k
 8      C[i] = C[i] + C[i − 1]
 9  // C[i] now contains the number of elements less than or equal to i.
10  // Copy A to B, starting from the end of A.
11  for j = n downto 1
12      B[C[A[j]]] = A[j]
13      C[A[j]] = C[A[j]] − 1    // to handle duplicate values
14  return B
```

C

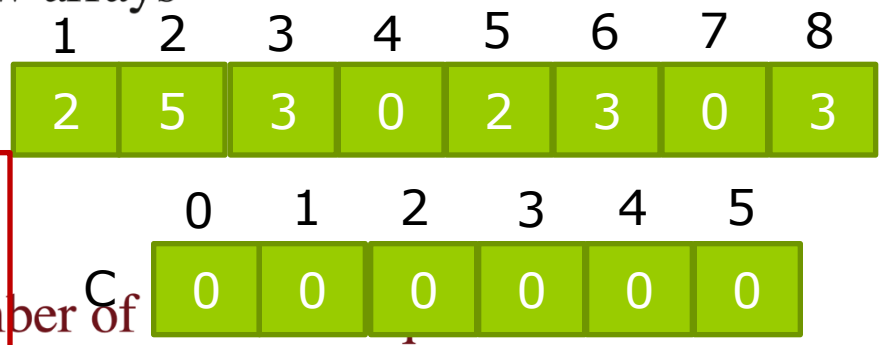
0	0	0	0	0	0

A

1	2	3	4	5	6	7	8
2	5	3	0	2	3	0	3

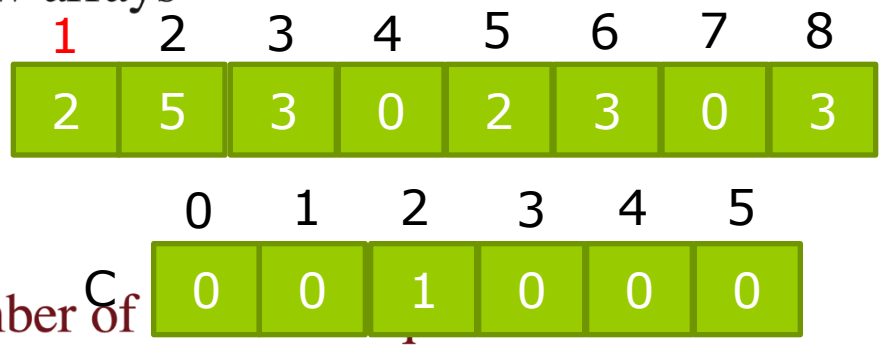
Sorting in linear time: Counting sort

```
COUNTING-SORT(A, n, k)
1   let B[1:n] and C[0:k] be new arrays
2   for i = 0 to k
3       C[i] = 0
4   for j = 1 to n
5       C[A[j]] = C[A[j]] + 1
6   // C[i] now contains the number of
7   for i = 1 to k
8       C[i] = C[i] + C[i − 1]
9   // C[i] now contains the number of elements less than or equal to i.
10  // Copy A to B, starting from the end of A.
11  for j = n downto 1
12      B[C[A[j]]] = A[j]
13      C[A[j]] = C[A[j]] − 1    // to handle duplicate values
14  return B
```



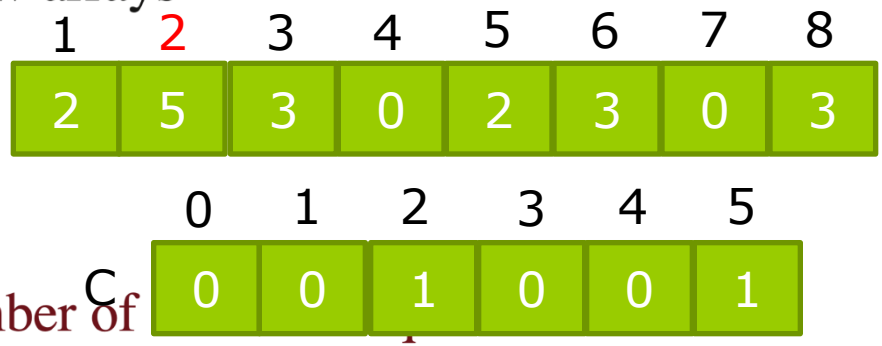
Sorting in linear time: Counting sort

```
COUNTING-SORT(A, n, k)
1   let B[1:n] and C[0:k] be new arrays
2   for i = 0 to k
3       C[i] = 0
4   for j = 1 to n
5       C[A[j]] = C[A[j]] + 1
6   // C[i] now contains the number of
7   for i = 1 to k
8       C[i] = C[i] + C[i − 1]
9   // C[i] now contains the number of elements less than or equal to i.
10  // Copy A to B, starting from the end of A.
11  for j = n downto 1
12      B[C[A[j]]] = A[j]
13      C[A[j]] = C[A[j]] − 1    // to handle duplicate values
14  return B
```



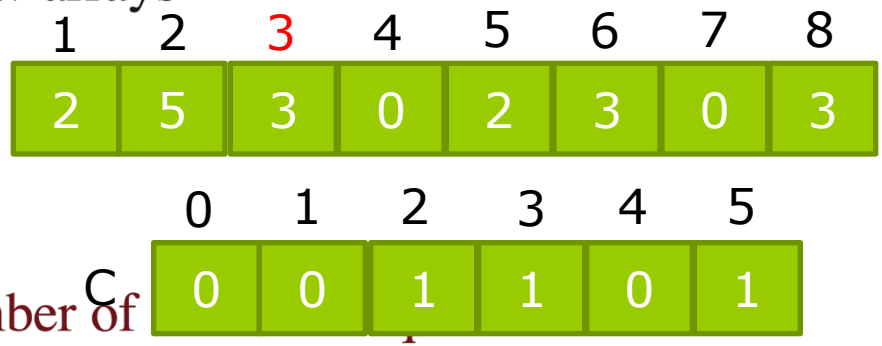
Sorting in linear time: Counting sort

```
COUNTING-SORT(A, n, k)
1   let B[1:n] and C[0:k] be new arrays
2   for i = 0 to k
3       C[i] = 0
4   for j = 1 to n
5       C[A[j]] = C[A[j]] + 1
6   // C[i] now contains the number of
7   for i = 1 to k
8       C[i] = C[i] + C[i − 1]
9   // C[i] now contains the number of elements less than or equal to i.
10  // Copy A to B, starting from the end of A.
11  for j = n downto 1
12      B[C[A[j]]] = A[j]
13      C[A[j]] = C[A[j]] − 1    // to handle duplicate values
14  return B
```



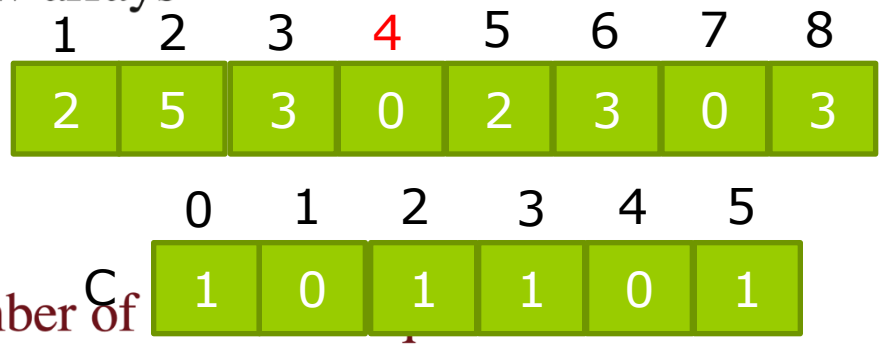
Sorting in linear time: Counting sort

```
COUNTING-SORT(A, n, k)
1   let B[1:n] and C[0:k] be new arrays
2   for i = 0 to k
3       C[i] = 0
4   for j = 1 to n
5       C[A[j]] = C[A[j]] + 1
6   // C[i] now contains the number of
7   for i = 1 to k
8       C[i] = C[i] + C[i − 1]
9   // C[i] now contains the number of elements less than or equal to i.
10  // Copy A to B, starting from the end of A.
11  for j = n downto 1
12      B[C[A[j]]] = A[j]
13      C[A[j]] = C[A[j]] − 1    // to handle duplicate values
14  return B
```



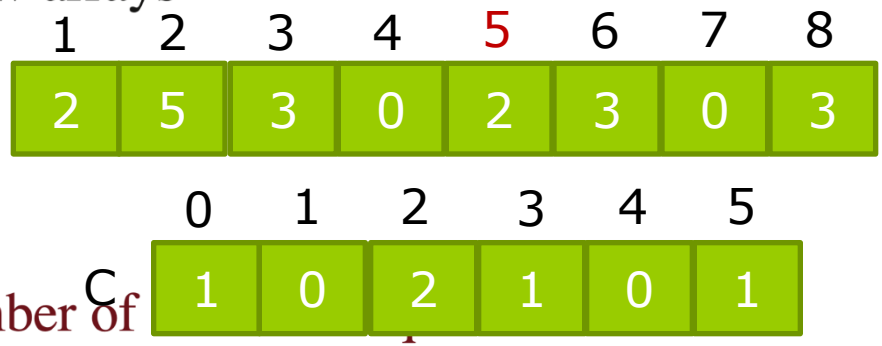
Sorting in linear time: Counting sort

```
COUNTING-SORT(A, n, k)
1   let B[1:n] and C[0:k] be new arrays
2   for i = 0 to k
3       C[i] = 0
4   for j = 1 to n
5       C[A[j]] = C[A[j]] + 1
6   // C[i] now contains the number of
7   for i = 1 to k
8       C[i] = C[i] + C[i − 1]
9   // C[i] now contains the number of elements less than or equal to i.
10  // Copy A to B, starting from the end of A.
11  for j = n downto 1
12      B[C[A[j]]] = A[j]
13      C[A[j]] = C[A[j]] − 1    // to handle duplicate values
14  return B
```



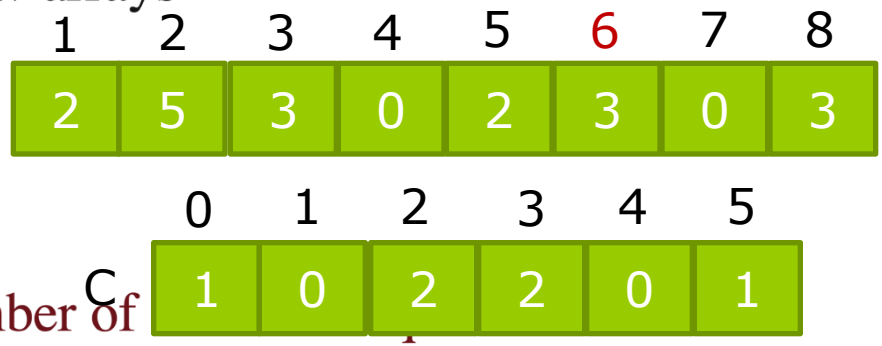
Sorting in linear time: Counting sort

```
COUNTING-SORT(A, n, k)
1   let B[1:n] and C[0:k] be new arrays
2   for i = 0 to k
3       C[i] = 0
4   for j = 1 to n
5       C[A[j]] = C[A[j]] + 1
6   // C[i] now contains the number of
7   for i = 1 to k
8       C[i] = C[i] + C[i − 1]
9   // C[i] now contains the number of elements less than or equal to i.
10  // Copy A to B, starting from the end of A.
11  for j = n downto 1
12      B[C[A[j]]] = A[j]
13      C[A[j]] = C[A[j]] − 1    // to handle duplicate values
14  return B
```



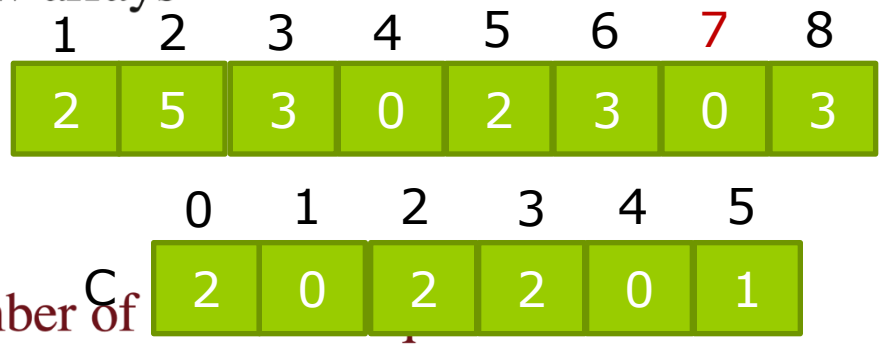
Sorting in linear time: Counting sort

```
COUNTING-SORT(A, n, k)
1   let B[1:n] and C[0:k] be new arrays
2   for i = 0 to k
3       C[i] = 0
4   for j = 1 to n
5       C[A[j]] = C[A[j]] + 1
6   // C[i] now contains the number of
7   for i = 1 to k
8       C[i] = C[i] + C[i − 1]
9   // C[i] now contains the number of elements less than or equal to i.
10  // Copy A to B, starting from the end of A.
11  for j = n downto 1
12      B[C[A[j]]] = A[j]
13      C[A[j]] = C[A[j]] − 1    // to handle duplicate values
14  return B
```



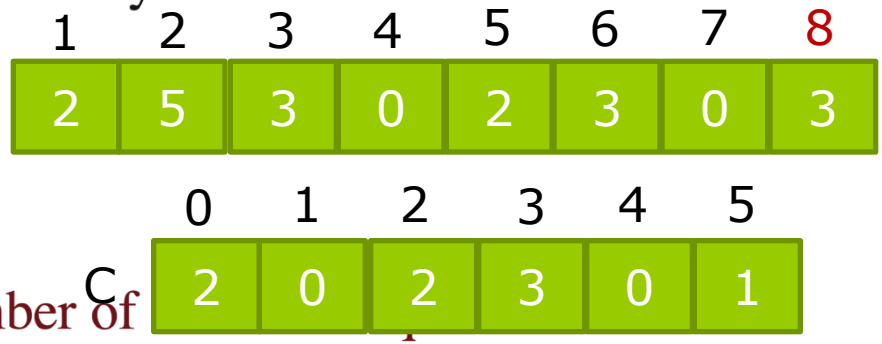
Sorting in linear time: Counting sort

```
COUNTING-SORT(A, n, k)
1   let B[1:n] and C[0:k] be new arrays
2   for i = 0 to k
3       C[i] = 0
4   for j = 1 to n
5       C[A[j]] = C[A[j]] + 1
6   // C[i] now contains the number of
7   for i = 1 to k
8       C[i] = C[i] + C[i − 1]
9   // C[i] now contains the number of elements less than or equal to i.
10  // Copy A to B, starting from the end of A.
11  for j = n downto 1
12      B[C[A[j]]] = A[j]
13      C[A[j]] = C[A[j]] − 1    // to handle duplicate values
14  return B
```



Sorting in linear time: Counting sort

```
COUNTING-SORT(A, n, k)
1   let B[1:n] and C[0:k] be new arrays
2   for i = 0 to k
3       C[i] = 0
4   for j = 1 to n
5       C[A[j]] = C[A[j]] + 1
6   // C[i] now contains the number of
7   for i = 1 to k
8       C[i] = C[i] + C[i − 1]
9   // C[i] now contains the number of elements less than or equal to i.
10  // Copy A to B, starting from the end of A.
11  for j = n downto 1
12      B[C[A[j]]] = A[j]
13      C[A[j]] = C[A[j]] − 1    // to handle duplicate values
14  return B
```



Sorting in linear time: Counting sort

```
COUNTING-SORT(A, n, k)
1   let B[1 : n] and C[0 : k] be new arrays
2   for i = 0 to k
3       C[i] = 0
4   for j = 1 to n
5       C[A[j]] = C[A[j]] + 1
6   // C[i] now contains the number of elements equal to i.
7   for i = 1 to k
8       C[i] = C[i] + C[i − 1]
9   // C[i] now contains the number of elements less than or equal to i.
10  // Copy A to B, starting from the end of A.
11  for j = n downto 1
12      B[C[A[j]]] = A[j]
13      C[A[j]] = C[A[j]] − 1    // to handle duplicate values
14  return B
```

A

1	2	3	4	5	6	7	8
2	5	3	0	2	3	0	3

C

0	1	2	3	4	5
2	0	2	3	0	1

Sorting in linear time: Counting sort

```
COUNTING-SORT(A, n, k)
1   let B[1:n] and C[0:k] be new arrays
2   for i = 0 to k
3       C[i] = 0
4   for j = 1 to n
5       C[A[j]] = C[A[j]] + 1
6   // C[i] now contains the number of elements equal to i.
7   for i = 1 to k
8       C[i] = C[i] + C[i − 1]
9   // C[i] now contains the number of elements less than or equal to i.
10  // Copy A to B, starting from the end of A.
11  for j = n downto 1
12      B[C[A[j]]] = A[j]
13      C[A[j]] = C[A[j]] − 1    // to handle duplicate values
14  return B
```

A

1	2	3	4	5	6	7	8
2	5	3	0	2	3	0	3

C

0	1	2	3	4	5
2	2	2	3	0	1

Sorting in linear time: Counting sort

```
COUNTING-SORT(A, n, k)
1   let B[1:n] and C[0:k] be new arrays
2   for i = 0 to k
3       C[i] = 0
4   for j = 1 to n
5       C[A[j]] = C[A[j]] + 1
6   // C[i] now contains the number of elements equal to i.
7   for i = 1 to k
8       C[i] = C[i] + C[i − 1]
9   // C[i] now contains the number of elements less than or equal to i.
10  // Copy A to B, starting from the end of A.
11  for j = n downto 1
12      B[C[A[j]]] = A[j]
13      C[A[j]] = C[A[j]] − 1    // to handle duplicate values
14  return B
```

A

1	2	3	4	5	6	7	8
2	5	3	0	2	3	0	3

C

0	1	2	3	4	5
2	2	4	3	0	1

Sorting in linear time: Counting sort

```
COUNTING-SORT(A, n, k)
1   let B[1:n] and C[0:k] be new arrays
2   for i = 0 to k
3       C[i] = 0
4   for j = 1 to n
5       C[A[j]] = C[A[j]] + 1
6   // C[i] now contains the number of elements equal to i.
7   for i = 1 to k
8       C[i] = C[i] + C[i − 1]
9   // C[i] now contains the number of elements less than or equal to i.
10  // Copy A to B, starting from the end of A.
11  for j = n downto 1
12      B[C[A[j]]] = A[j]
13      C[A[j]] = C[A[j]] − 1    // to handle duplicate values
14  return B
```

A

1	2	3	4	5	6	7	8
2	5	3	0	2	3	0	3

C

0	1	2	3	4	5
2	2	4	7	0	1

Sorting in linear time: Counting sort

```
COUNTING-SORT(A, n, k)
1   let B[1:n] and C[0:k] be new arrays
2   for i = 0 to k
3       C[i] = 0
4   for j = 1 to n
5       C[A[j]] = C[A[j]] + 1
6   // C[i] now contains the number of elements equal to i.
7   for i = 1 to k
8       C[i] = C[i] + C[i − 1]
9   // C[i] now contains the number of elements less than or equal to i.
10  // Copy A to B, starting from the end of A.
11  for j = n downto 1
12      B[C[A[j]]] = A[j]
13      C[A[j]] = C[A[j]] − 1    // to handle duplicate values
14  return B
```

A

1	2	3	4	5	6	7	8
2	5	3	0	2	3	0	3

C

0	1	2	3	4	5
2	2	4	7	7	1

Sorting in linear time: Counting sort

```
COUNTING-SORT(A, n, k)
1   let B[1:n] and C[0:k] be new arrays
2   for i = 0 to k
3       C[i] = 0
4   for j = 1 to n
5       C[A[j]] = C[A[j]] + 1
6   // C[i] now contains the number of elements equal to i.
7   for i = 1 to k
8       C[i] = C[i] + C[i − 1]
9   // C[i] now contains the number of elements less than or equal to i.
10  // Copy A to B, starting from the end of A.
11  for j = n downto 1
12      B[C[A[j]]] = A[j]
13      C[A[j]] = C[A[j]] − 1    // to handle duplicate values
14  return B
```

A

1	2	3	4	5	6	7	8
2	5	3	0	2	3	0	3

C

0	1	2	3	4	5
2	2	4	7	7	8

Sorting in linear time: Counting sort

```
COUNTING-SORT(A, n, k)
1   let B[1:n] and C[0:k] be new arrays
2   for i = 0 to k
3       C[i] = 0
4   for j = 1 to n
5       C[A[j]] = C[A[j]] + 1
6   // C[i] now contains the number of elements equal to i.
7   for i = 1 to k
8       C[i] = C[i] + C[i − 1]
9   // C[i] now contains the number of elements less than or equal to i.
10  // Copy A to B, starting from the end of A.
11  for j = n downto 1
12      B[C[A[j]]] = A[j]
13      C[A[j]] = C[A[j]] − 1
14  return B
```

A

1	2	3	4	5	6	7	8
2	5	3	0	2	3	0	3

C

0	1	2	3	4	5
2	2	4	7	7	8

B

1	2	3	4	5	6	7	8

// to handle duplicate values

Sorting in linear time: Counting sort

```
COUNTING-SORT(A, n, k)
1   let B[1:n] and C[0:k] be new arrays
2   for i = 0 to k
3       C[i] = 0
4   for j = 1 to n
5       C[A[j]] = C[A[j]] + 1
6   // C[i] now contains the number of elements equal to i.
7   for i = 1 to k
8       C[i] = C[i] + C[i − 1]
9   // C[i] now contains the number of elements less than or equal to i.
10  // Copy A to B, starting from the end of A.
11  for j = n downto 1
12      B[C[A[j]]] = A[j]
13      C[A[j]] = C[A[j]] − 1
14  return B
```

A

1	2	3	4	5	6	7	8
2	5	3	0	2	3	0	3

C

0	1	2	3	4	5
2	2	4	7	7	8

B

1	2	3	4	5	6	7	8
						3	

// to handle duplicate values

Sorting in linear time: Counting sort

```
COUNTING-SORT(A, n, k)
1   let B[1:n] and C[0:k] be new arrays
2   for i = 0 to k
3       C[i] = 0
4   for j = 1 to n
5       C[A[j]] = C[A[j]] + 1
6   // C[i] now contains the number of elements equal to i.
7   for i = 1 to k
8       C[i] = C[i] + C[i − 1]
9   // C[i] now contains the number of elements less than or equal to i.
10  // Copy A to B, starting from the end of A.
11  for j = n downto 1
12      B[C[A[j]]] = A[j]
13      C[A[j]] = C[A[j]] − 1
14  return B
```

A

1	2	3	4	5	6	7	8
2	5	3	0	2	3	0	3

C

0	1	2	3	4	5
2	2	4	6	7	8

B

1	2	3	4	5	6	7	8
						3	

// to handle duplicate values

Sorting in linear time: Counting sort

```
COUNTING-SORT(A, n, k)
1   let B[1:n] and C[0:k] be new arrays
2   for i = 0 to k
3       C[i] = 0
4   for j = 1 to n
5       C[A[j]] = C[A[j]] + 1
6   // C[i] now contains the number of elements equal to i.
7   for i = 1 to k
8       C[i] = C[i] + C[i − 1]
9   // C[i] now contains the number of elements less than or equal to i.
10  // Copy A to B, starting from the end of A.
11  for j = n downto 1
12      B[C[A[j]]] = A[j]
13      C[A[j]] = C[A[j]] − 1
14  return B
```

A

1	2	3	4	5	6	7	8
2	5	3	0	2	3	0	3

C

0	1	2	3	4	5
2	2	4	6	7	8

B

1	2	3	4	5	6	7	8
	0					3	

// to handle duplicate values

Sorting in linear time: Counting sort

```
COUNTING-SORT(A, n, k)
1   let B[1:n] and C[0:k] be new arrays
2   for i = 0 to k
3       C[i] = 0
4   for j = 1 to n
5       C[A[j]] = C[A[j]] + 1
6   // C[i] now contains the number of elements equal to i.
7   for i = 1 to k
8       C[i] = C[i] + C[i − 1]
9   // C[i] now contains the number of elements less than or equal to i.
10  // Copy A to B, starting from the end of A.
11  for j = n downto 1
12      B[C[A[j]]] = A[j]
13      C[A[j]] = C[A[j]] − 1
14  return B
```

A

1	2	3	4	5	6	7	8
2	5	3	0	2	3	0	3

C

0	1	2	3	4	5
1	2	4	6	7	8

B

1	2	3	4	5	6	7	8
	0					3	

// to handle duplicate values

Sorting in linear time: Counting sort

```
COUNTING-SORT(A, n, k)
1  let B[1:n] and C[0:k] be new arrays
2  for i = 0 to k
3      C[i] = 0
4  for j = 1 to n
5      C[A[j]] = C[A[j]] + 1
6  // C[i] now contains the number of elements equal to i.
7  for i = 1 to k
8      C[i] = C[i] + C[i − 1]
9  // C[i] now contains the number of elements less than or equal to i.
10 // Copy A to B, starting from the end of A.
11 for j = n downto 1
12     B[C[A[j]]] = A[j]
13     C[A[j]] = C[A[j]] − 1
14 return B
```

A

1	2	3	4	5	6	7	8
2	5	3	0	2	3	0	3

C

0	1	2	3	4	5
1	2	4	6	7	8

B

1	2	3	4	5	6	7	8
	0				3	3	

// to handle duplicate values

Sorting in linear time: Counting sort

```
COUNTING-SORT(A, n, k)
1   let B[1:n] and C[0:k] be new arrays
2   for i = 0 to k
3       C[i] = 0
4   for j = 1 to n
5       C[A[j]] = C[A[j]] + 1
6   // C[i] now contains the number of elements equal to i.
7   for i = 1 to k
8       C[i] = C[i] + C[i − 1]
9   // C[i] now contains the number of elements less than or equal to i.
10  // Copy A to B, starting from the end of A.
11  for j = n downto 1
12      B[C[A[j]]] = A[j]
13      C[A[j]] = C[A[j]] − 1
14  return B
```

A

1	2	3	4	5	6	7	8
2	5	3	0	2	3	0	3

C

0	1	2	3	4	5
1	2	4	5	7	8

B

1	2	3	4	5	6	7	8
	0				3	3	

// to handle duplicate values

Sorting in linear time: Counting sort

```
COUNTING-SORT(A, n, k)
1   let B[1:n] and C[0:k] be new arrays
2   for i = 0 to k
3       C[i] = 0
4   for j = 1 to n
5       C[A[j]] = C[A[j]] + 1
6   // C[i] now contains the number of elements equal to i.
7   for i = 1 to k
8       C[i] = C[i] + C[i − 1]
9   // C[i] now contains the number of elements less than or equal to i.
10  // Copy A to B, starting from the end of A.
11  for j = n downto 1
12      B[C[A[j]]] = A[j]
13      C[A[j]] = C[A[j]] − 1
14  return B
```

A

1	2	3	4	5	6	7	8
2	5	3	0	2	3	0	3

C

0	1	2	3	4	5
1	2	4	5	7	8

B

1	2	3	4	5	6	7	8
	0		2		3	3	

// to handle duplicate values

Sorting in linear time: Counting sort

```
COUNTING-SORT(A, n, k)
1   let B[1:n] and C[0:k] be new arrays
2   for i = 0 to k
3       C[i] = 0
4   for j = 1 to n
5       C[A[j]] = C[A[j]] + 1
6   // C[i] now contains the number of elements equal to i.
7   for i = 1 to k
8       C[i] = C[i] + C[i − 1]
9   // C[i] now contains the number of elements less than or equal to i.
10  // Copy A to B, starting from the end of A.
11  for j = n downto 1
12      B[C[A[j]]] = A[j]
13      C[A[j]] = C[A[j]] − 1
14  return B
```

A

1	2	3	4	5	6	7	8
2	5	3	0	2	3	0	3

C

0	1	2	3	4	5
1	2	3	5	7	8

B

1	2	3	4	5	6	7	8
	0		2		3	3	

// to handle duplicate values

Sorting in linear time: Counting sort

```
COUNTING-SORT(A, n, k)
1   let B[1:n] and C[0:k] be new arrays
2   for i = 0 to k
3       C[i] = 0
4   for j = 1 to n
5       C[A[j]] = C[A[j]] + 1
6   // C[i] now contains the number of elements equal to i.
7   for i = 1 to k
8       C[i] = C[i] + C[i − 1]
9   // C[i] now contains the number of elements less than or equal to i.
10  // Copy A to B, starting from the end of A.
11  for j = n downto 1
12      B[C[A[j]]] = A[j]
13      C[A[j]] = C[A[j]] − 1
14  return B
```

A

1	2	3	4	5	6	7	8
2	5	3	0	2	3	0	3

C

0	1	2	3	4	5
1	2	3	5	7	8

B

1	2	3	4	5	6	7	8
0	0		2		3	3	

// to handle duplicate values

Sorting in linear time: Counting sort

```
COUNTING-SORT(A, n, k)
1   let B[1:n] and C[0:k] be new arrays
2   for i = 0 to k
3       C[i] = 0
4   for j = 1 to n
5       C[A[j]] = C[A[j]] + 1
6   // C[i] now contains the number of elements equal to i.
7   for i = 1 to k
8       C[i] = C[i] + C[i − 1]
9   // C[i] now contains the number of elements less than or equal to i.
10  // Copy A to B, starting from the end of A.
11  for j = n downto 1
12      B[C[A[j]]] = A[j]
13      C[A[j]] = C[A[j]] − 1
14  return B
```

A

1	2	3	4	5	6	7	8
2	5	3	0	2	3	0	3

C

0	1	2	3	4	5
0	2	3	5	7	8

B

1	2	3	4	5	6	7	8
0	0		2		3	3	

// to handle duplicate values

Sorting in linear time: Counting sort

```
COUNTING-SORT(A, n, k)
1   let B[1:n] and C[0:k] be new arrays
2   for i = 0 to k
3       C[i] = 0
4   for j = 1 to n
5       C[A[j]] = C[A[j]] + 1
6   // C[i] now contains the number of elements equal to i.
7   for i = 1 to k
8       C[i] = C[i] + C[i − 1]
9   // C[i] now contains the number of elements less than or equal to i.
10  // Copy A to B, starting from the end of A.
11  for j = n downto 1
12      B[C[A[j]]] = A[j]
13      C[A[j]] = C[A[j]] − 1
14  return B
```

A

1	2	3	4	5	6	7	8
2	5	3	0	2	3	0	3

C

0	1	2	3	4	5
0	2	3	5	7	8

B

1	2	3	4	5	6	7	8
0	0		2	3	3	3	

// to handle duplicate values

Sorting in linear time: Counting sort

```
COUNTING-SORT(A, n, k)
1   let B[1:n] and C[0:k] be new arrays
2   for i = 0 to k
3       C[i] = 0
4   for j = 1 to n
5       C[A[j]] = C[A[j]] + 1
6   // C[i] now contains the number of elements equal to i.
7   for i = 1 to k
8       C[i] = C[i] + C[i − 1]
9   // C[i] now contains the number of elements less than or equal to i.
10  // Copy A to B, starting from the end of A.
11  for j = n downto 1
12      B[C[A[j]]] = A[j]
13      C[A[j]] = C[A[j]] − 1
14  return B
```

A

1	2	3	4	5	6	7	8
2	5	3	0	2	3	0	3

C

0	1	2	3	4	5
0	2	3	4	7	8

B

1	2	3	4	5	6	7	8
0	0		2	3	3	3	

// to handle duplicate values

Sorting in linear time: Counting sort

```
COUNTING-SORT(A, n, k)
1   let B[1:n] and C[0:k] be new arrays
2   for i = 0 to k
3       C[i] = 0
4   for j = 1 to n
5       C[A[j]] = C[A[j]] + 1
6   // C[i] now contains the number of elements equal to i.
7   for i = 1 to k
8       C[i] = C[i] + C[i − 1]
9   // C[i] now contains the number of elements less than or equal to i.
10  // Copy A to B, starting from the end of A.
11  for j = n downto 1
12      B[C[A[j]]] = A[j]
13      C[A[j]] = C[A[j]] − 1
14  return B
```

A

1	2	3	4	5	6	7	8
2	5	3	0	2	3	0	3

C

0	1	2	3	4	5
0	2	3	4	7	8

B

1	2	3	4	5	6	7	8
0	0		2	3	3	3	5

// to handle duplicate values

Sorting in linear time: Counting sort

```
COUNTING-SORT(A, n, k)
1   let B[1:n] and C[0:k] be new arrays
2   for i = 0 to k
3       C[i] = 0
4   for j = 1 to n
5       C[A[j]] = C[A[j]] + 1
6   // C[i] now contains the number of elements equal to i.
7   for i = 1 to k
8       C[i] = C[i] + C[i − 1]
9   // C[i] now contains the number of elements less than or equal to i.
10  // Copy A to B, starting from the end of A.
11  for j = n downto 1
12      B[C[A[j]]] = A[j]
13      C[A[j]] = C[A[j]] − 1
14  return B
```

A

1	2	3	4	5	6	7	8
2	5	3	0	2	3	0	3

C

0	1	2	3	4	5
0	2	3	4	7	7

B

1	2	3	4	5	6	7	8
0	0		2	3	3	3	5

// to handle duplicate values

Sorting in linear time: Counting sort

```
COUNTING-SORT(A, n, k)
1   let B[1:n] and C[0:k] be new arrays
2   for i = 0 to k
3       C[i] = 0
4   for j = 1 to n
5       C[A[j]] = C[A[j]] + 1
6   // C[i] now contains the number of elements equal to i.
7   for i = 1 to k
8       C[i] = C[i] + C[i − 1]
9   // C[i] now contains the number of elements less than or equal to i.
10  // Copy A to B, starting from the end of A.
11  for j = n downto 1
12      B[C[A[j]]] = A[j]
13      C[A[j]] = C[A[j]] − 1
14  return B
```

A

1	2	3	4	5	6	7	8
2	5	3	0	2	3	0	3

C

0	1	2	3	4	5
0	2	3	4	7	7

B

1	2	3	4	5	6	7	8
0	0	2	2	3	3	3	5

// to handle duplicate values

Sorting in linear time: Counting sort

```
COUNTING-SORT(A, n, k)
1   let B[1:n] and C[0:k] be new arrays
2   for i = 0 to k
3       C[i] = 0
4   for j = 1 to n
5       C[A[j]] = C[A[j]] + 1
6   // C[i] now contains the number of elements equal to i.
7   for i = 1 to k
8       C[i] = C[i] + C[i − 1]
9   // C[i] now contains the number of elements less than or equal to i.
10  // Copy A to B, starting from the end of A.
11  for j = n downto 1
12      B[C[A[j]]] = A[j]
13      C[A[j]] = C[A[j]] − 1
14  return B
```

A

1	2	3	4	5	6	7	8
2	5	3	0	2	3	0	3

C

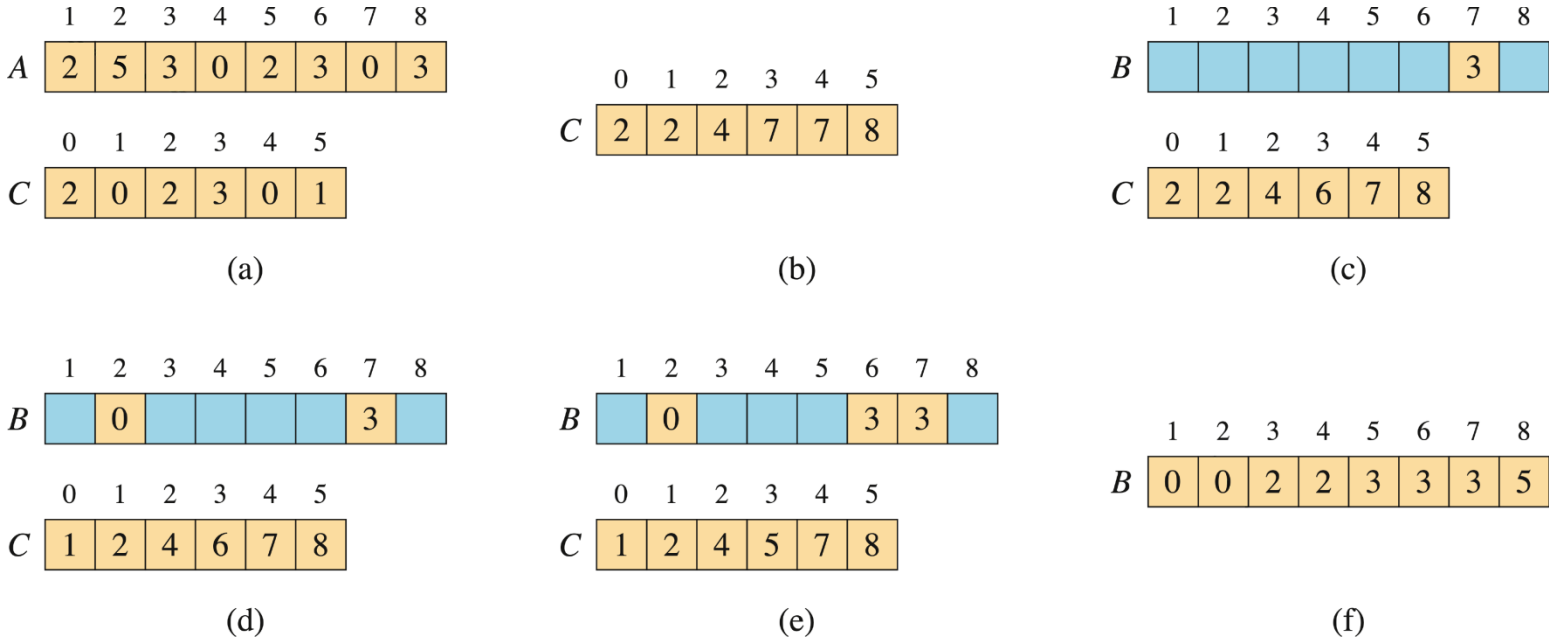
0	1	2	3	4	5
0	2	2	4	7	7

B

1	2	3	4	5	6	7	8
0	0	2	2	3	3	3	5

// to handle duplicate values

Sorting in linear time: Counting sort



- The operation of COUNTING-SORT on an input $A[1 : 8]$, where each element of A is a nonnegative integer no larger than $k = 5$.

  (a) The array A and the auxiliary array C after line 5 (after the second **for** loop).

  (b) The array C after line 8 (after the third **for** loop).

  (c)-(e) The output array B and the auxiliary array C after one, two, and three iterations of the loop in lines 11-13, respectively. Only the tan elements of array B have been filled in.

  (f) The final sorted output array B.

Sorting in linear time: Counting sort

- For example, $A = \langle 2_1, 5_1, 3_1, 0_1, 2_2, 3_2, 0_2, 3_3 \rangle$ (Subscripts show original order of equal keys in order to demonstrate stability.)

i	0	1	2	3	4	5
$C[i]$ after second **for** loop	2	0	2	3	0	1
$C[i]$ after third **for** loop	2	2	4	7	7	8

- Sorted output is $\langle 0_1, 0_2, 2_1, 2_2, 3_1, 3_2, 3_3, 5_1 \rangle$.

- ***Idea***: After the third **for** loop, $C[i]$ counts how many keys are less than or equal to i. If all elements are distinct, then an element with value i should go into $B[i]$. But if elements are not distinct, by examining values in A in reverse order, the last **for** loop puts $A[j]$ into $B[C[A[j]]]$ and then decrements $C[A[j]]$ so that the next time it finds elements with the same value as $A[j]$, that element goes into the position of B just before $A[j]$.

Sorting in linear time: Counting sort – Analysis

- Counting sort is ***stable*** (keys with the same value appear in the same order in the output as they do in the input) because of how the last loop works.

- ***Analysis***
  - The overall time is $\Theta(n + k)$.
  - In practice, we usually use counting sort when we have $k = O(n)$, in which case the running time is $\Theta(n)$.
  - Counting sort beats the lower bound of $\Omega(n \lg n)$ because it is not a comparison sort.
  - In fact, no comparisons between input elements occur anywhere in the code.

Reading

- Sections 8.1 ~ 8.2